Student Name: _____

Student Email: _____

**Homework 3 Quiz (CS 7150)**
Fall 2026

---

## Instructions

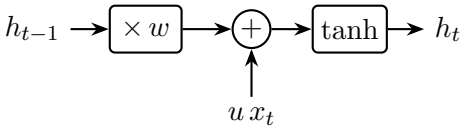
- Write your name and your Northeastern email address on every page.
- You have 15 minutes. Write your answers in the spaces directly after each question.
- For worked questions, draw a single BOX around your final answer, so that graders can tell the difference between your answer and your scratch work.
- Closed book: no notes, books, computers, calculators, or other resources, and no communication with anyone other than the proctor.
- You may not share or discuss these questions with any student who has not yet taken the quiz.

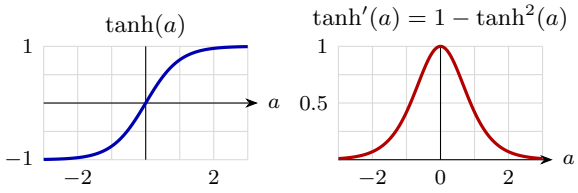
I attest that I have read and will follow the closed-book rules above.

Student signature: _____ Date: _____

1. (2 points) Backpropagation through time in a recurrent network. Suppose we have a recurrent network made of tiny Elman cells each with a single number $h_t$ as its hidden state:

$$h_t = \tanh(w\, h_{t-1} + u\, x_t), \qquad w = 0.5.$$




The chain rule through the tanh gives $\dfrac{\partial h_t}{\partial h_{t-1}} = w\,(1-h_t^2)$, using $\tanh'(a) = 1-\tanh^2(a)$, which is never more than 1 (plotted above).

Backpropagation through time (BPTT) multiplies one such factor per step. Consider BPTT from the 10th step. What is the largest that $\dfrac{\partial h_{10}}{\partial h_0}$ can possibly be? A fraction is fine.

$$\frac{\partial h_{10}}{\partial h_0} \leq \boxed{\phantom{00}}$$

**Solution:** Each factor $w\,(1 - h_t^2)$ is at most $w = 0.5$, since $0 < 1 - h_t^2 \leq 1$, so $\dfrac{\partial h_{10}}{\partial h_0} \leq 0.5^{10} = \dfrac{1}{1024} \approx 0.001$. Whatever $h_0$ held has almost no influence on the gradient ten steps later: the gradient vanishes. (For comparison, a GRU whose update gate is held at $z_t = 0.9$ keeps $0.9^{10} \approx 0.35$.)

Grading: 1 point for the product of ten factors, each bounded by $w$; 1 for the value.

Student Name: _____ Student Email: _____

2. (4 points) True or false, circle one.

   (a) An Elman RNN trained on sequences of length 100 has more parameters than the same RNN trained on sequences of length 10. True False

   **Solution: False.** The same weights $W_{xh}$, $W_{hh}$ and $W_o$ are applied at every time step, so the parameter count does not depend on the sequence length. Weight sharing across time is what lets one network handle sequences of any length, just as a convolution shares its weights across positions.

   (b) During training, an RNN language model that uses a GRU can compute its hidden states for all the positions of a sequence in parallel, the way a transformer does. True False

   **Solution: False.** Each $h_t$ needs $h_{t-1}$, so the RNN must run one step after another. A transformer computes every position's output at once, because each position only needs the (known) inputs, and the causal mask keeps it from using the future. This is the main practical reason transformers displaced RNNs: they use a GPU's parallelism during training.

   (c) When generating text one token at a time, the work to produce each new token stays the same for an RNN, but grows with the length of the text so far for a transformer. True False

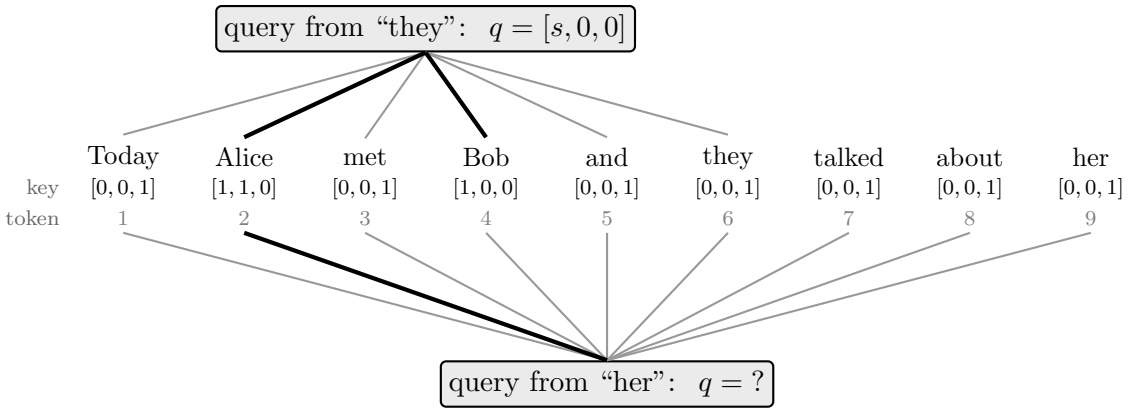
   **Solution: True.** The RNN updates one fixed-size state per token. The transformer's new token attends over every earlier position, so the cost of token $t$ grows like $t$. The transformer pays for direct access to the whole past; the RNN gets cheap steps but must squeeze the past into one vector.

   (d) A self-attention layer with no position embeddings and no causal mask: if the input tokens are shuffled, its outputs are shuffled the same way, and otherwise unchanged. True False

   **Solution: True.** Attention treats its inputs as a set: each output is a weighted average whose weights depend only on query–key dot products, not on where the tokens sit. So without position embeddings "dog bites man" and "man bites dog" look the same; that is why the HW3 transformer adds a learned position embedding to each token embedding.

3. Finding a name by content in a transformer. A language model is reading "Today Alice met Bob and they talked about her ...". One attention head looks back for the people that a pronoun refers to. Each token's key is computed from its word embedding, and in this head it comes out as three numbers: [is a name, is a female name, is an ordinary word]. So Alice has key $[1, 1, 0]$, Bob has key $[1, 0, 0]$, and every other token, the pronouns included, has key $[0, 0, 1]$. Each token $j$ gets the score $\text{score}_j = q \cdot k_j$, the dot product of the query $q$ with its key, and the attention weights are the softmax of the scores, $a_j = e^{\text{score}_j} / \sum_k e^{\text{score}_k}$, over the tokens the causal mask allows: the current token and the ones before it.

Student Name: _____ Student Email: _____




(a) (2 points) At "they" (token 6), the head sees tokens 1 to 6. Suppose the query from "they" is $q = [s, 0, 0]$, which looks only for names, with $e^s = 4$. What fraction of the attention goes to Alice, to Bob, and to each of the other four tokens?

Alice: $a_2 =$ ☐ Bob: $a_4 =$ ☐ other tokens e.g.,: $a_1 =$ ☐

**Solution:** The scores are $s$ for Alice and Bob and 0 for "Today", "met", "and" and "they", so the weights are $\frac{4}{4+4+4\cdot 1} = \frac{1}{3}$ for Alice, $\frac{1}{3}$ for Bob, and $\frac{1}{12}$ for each other token. The query for "they" finds both people; it does not need to tell them apart.

Grading: 1 point for the two names, 1 for $\frac{1}{12}$.

(b) (2 points) At "her" (token 9), the head sees all nine tokens. Find a query $q = [q_1, q_2, q_3]$ from "her" that gives $a_2 = \frac{1}{2}$ of the attention to Alice and $a_j = \frac{1}{16}$ to each of the other eight tokens $j \neq 2$, Bob included. You may leave logarithms in your answer, such as $\ln 2$.

$q =$ [ ☐ , ☐ , ☐ ]

**Solution:** Bob must get the same weight as the ordinary words, so his score $q_1$ must equal theirs, $q_3$; take $q_1 = q_3 = 0$. Alice must get 8 times the weight of each other token ($\frac{1}{2} = 8 \cdot \frac{1}{16}$), so $e^{q_1+q_2} = 8$ and $q_2 = \ln 8$:

$$q = [0,\ \ln 8,\ 0].$$

Check: $8 + 8 \cdot 1 = 16$, so Alice gets $\frac{8}{16} = \frac{1}{2}$ and every other token $\frac{1}{16}$. Where the query for "they" looked in the "name" direction, the query for "her" looks only in the "female name" direction, the one feature that separates Alice from Bob. Adding the same constant $c$ to $q_1$ and $q_3$ also leaves the softmax unchanged, so $[c,\ \ln 8,\ c]$ is also correct. There is no need to add $c$ to $q_2$: Alice's key $[1, 1, 0]$ has a 1 in the name slot, so her score $q_1 + q_2$ picks up the $c$ automatically through $q_1$. Bob's score $q_1$ and the others' score $q_3$ also go up by $c$, so every score shifts by the same $c$.

Grading: 1 point for $q_1 = q_3$ (Bob treated like an ordinary word), 1 for $q_2 = \ln 8$.